МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

# МЕТОДИЧНІ ВКАЗІВКИ
# ЩОДО ВИКОНАННЯ ПРАКТИЧНИХ І САМОСТІЙНОЇ РОБІТ
# З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
# **«АЛГОРИТМИ ТА МЕТОДИ ОБЧИСЛЕНЬ»**

ДЛЯ СТУДЕНТІВ ДЕННОЇ ФОРМИ НАВЧАННЯ
ЗІ СПЕЦІАЛЬНОСТІ 122 – «КОМП’ЮТЕРНІ НАУКИ»
ОСВІТНЬО-ПРОФЕСІЙНОЇ ПРОГРАМИ «КОМП’ЮТЕРНІ НАУКИ»
ОСВІТНЬОГО СТУПЕНЯ «БАКАЛАВР»
(ЧАСТИНА 1)

КРЕМЕНЧУК 2023

Методичні вказівки щодо виконання практичних і самостійної робіт з навчальної дисципліни «Алгоритми та методи обчислень» для студентів денної форми навчання зі спеціальності 122 – «Комп'ютерні науки» освітньо-професійної програми «Комп'ютерні науки» освітнього ступеня «Бакалавр» (частина 1)

Укладач к. т. н., доц. В. М. Сидоренко

Рецензент д. т. н., проф. М. І. Гученко

Кафедра комп'ютерної інженерії та електроніки

Затверджено методичною радою Кременчуцького національного університету імені Михайла Остроградського

Протокол _9_ від __28.06___ 2023 року

Голова методичної ради _____ проф. В. В. Костін

# ЗМІСТ

Вступ……. 4

1 Вимоги до оформлення звітів про виконання практичних робіт…… 6

2 Перелік практичних робіт……..…….. 7

Практична робота № 1 Асимптотична складність алгоритмів. О-нотація …….. 7

Практична робота № 2 Асимптотична складність алгоритмів. Інші нотації ……. 13

Практична робота № 3 Алгоритми сортування та їх складність. Порівняння алгоритмів сортування …….. 20

Практична робота № 4 Алгоритми пошуку та їх складність …… 27

3 Критерії оцінювання знань студентів…….. 31

Список літератури …….....……... 33

Додаток А Зразок оформлення титульної сторінки звіту…… 37

# ВСТУП

Навчальна дисципліна «Алгоритми та методи обчислень» належить до циклу навчальних дисциплін математичної підготовки бакалаврської програми.

Мета практичних занять – забезпечити закріплення студентами теоретичних знань, отриманих на лекціях, набути практичних навичок аналізу алгоритмів, розв'язання типових задач щодо структур даних та методів побудови ефективних алгоритмів, що передує написанню оптимального програмного коду.

Перша частина методичних вказівок уміщують короткі теоретичні відомості, приклади розв'язування задач, вимоги щодо оформлення звіту індивідуальних завдань в рамках першого змістового модуля «Алгоритми і структури даних».

Метою навчальної дисципліни є набуття студентами теоретичних знань і практичних навичок щодо методів побудови ефективних алгоритмів та обчислювальних методів для розв'язання прикладних і наукових завдань в області комп'ютерної інженерії.

Завданням навчальної дисципліни є вивчення основних структур даних, методів розробки та аналізу алгоритмів, вибору та обґрунтовування стійких до похибок методів обчислень.

У результаті вивчення навчальної дисципліни студент повинен

***отримати досвід з компетентностей:***

ЗК 1. Здатність до абстрактного мислення, аналізу та синтезу.

ЗК 2. Здатність застосовувати знання у практичних ситуаціях.

ЗК 3. Знання та розуміння предметної області та розуміння професійної діяльності.

ЗК 6. Здатність вчитися й оволодівати сучасними знаннями.

ЗК 7. Здатність до пошуку, оброблення та аналізу інформації з різних джерел.

ЗК 8. Здатність генерувати нові ідеї (креативність).

ЗК 11. Здатність приймати обґрунтовані рішення.

ЗК 12. Здатність оцінювати та забезпечувати якість виконуваних робіт.

СК 3. Здатність до логічного мислення, побудови логічних висновків, використання формальних мов і моделей алгоритмічних обчислень, проєктування, розроблення й аналізу алгоритмів, оцінювання їх ефективності та складності, розв'язності та нерозв'язності алгоритмічних проблем для адекватного моделювання предметних областей і створення програмних та інформаційних систем;

***набути навички та уміння:***

ПР 2. Використовувати сучасний математичний апарат неперервного та дискретного аналізу, лінійної алгебри, аналітичної геометрії, в професійній діяльності для розв'язання задач теоретичного та прикладного характеру в процесі проєктування та реалізації об'єктів інформатизації.

ПР 5. Проєктувати, розробляти та аналізувати алгоритми розв'язання обчислювальних та логічних задач, оцінювати ефективність та складність алгоритмів на основі застосування формальних моделей алгоритмів та обчислюваних функцій.

Згідно з вимогами освітньо-професійної програми студент повинен

**знати:**

– сучасні методи побудови та аналізу ефективних алгоритмів та способи їх реалізації в конкретних застосуваннях;

– базові структури даних та спосіб реалізації їх на мові Python;

– знати основні алгоритми роботи з даними: на рядках, на графах, оптимального кодування без втрат;

**уміти:**

– розробляти, аналізувати та застосовувати ефективні алгоритми для розв'язання професійних завдань в області інформаційних технологій;

– реалізовувати базові структури даних на мові програмування Python;

– реалізовувати основні алгоритми роботи з даними на мові Python.

# 1 ВИМОГИ ДО ОФОРМЛЕННЯ ЗВІТІВ З ПРАКТИЧНИХ РОБІТ

Звіт щодо виконання практичних робіт має бути написаний студентом від руки розбірливим почерком у зошиті в клітинку чи на комп'ютері шрифтом Times New Roman, розміром 14 пунктів на одному боці аркуша, через півтора інтервали. Відстань між попереднім текстом і заголовком має бути два інтервали, а відстань між заголовком і наступним текстом – у півтора рази більше, ніж міжрядковий проміжок звичайного тексту. Після заголовка на сторінці має бути хоча б один рядок тексту.

**Зміст звіту**

За підсумками кожної практичної роботи студент оформлює індивідуальний звіт, що містить:

– титульну сторінку;

– тему роботи;

– постановку завдання;

– розв'язання задачі згідно зі своїм варіантом;

– отримані результати;

– відповіді на контрольні питання.

## 2 ПЕРЕЛІК ПРАКТИЧНИХ РОБІТ

### Практична робота № 1

**Тема. Асимптотична складність алгоритмів. О-нотація**

**Мета:** набути практичних навичок у розв'язанні задач на оцінку асимптотичної складності алгоритмів у $O$.

**Короткі теоретичні відомості**

Припустимо, що ми написали алгоритм і перед нами стоїть задача оцінки його ефективності [41].

Коли справа доходить до аналізу ефективності алгоритму, ми зазвичай пишемо функцію, скажімо $f(n)$, яка представляє кількість часу, кількість кроків або кількість простору, необхідного для вирішення проблеми розміру $n$. У цьому випадку нас не цікавить точне значення $f$ для певного $n$, але нас цікавить, як швидко зростає $f(n)$ зі збільшенням $n$. Іншими словами, ми хочемо знати *швидкість зростання* функції $f$.

Для опису зростання функцій із збільшенням розміру входу, використовуються так звані *асимптотичні нотації*. Вони описують граничну поведінку функції, коли її аргумент прагне до нескінченності: $n \to \infty$.

Перша важлива з нотацій це так звана *велика О-нотація* (велика літера $O$, а не нуль), яке також називають символом Ландау. Буква $O$ використовується тому, що швидкість зростання функції також може називатися її порядком. На прикладі часу виконання алгоритму велике $O$ використовується як асимптотична *верхня межа* часу виконання.

*Визначення.* Нехай $f$ і $g$ – дві функції, що відображають натуральні у простір дійсних чисел. Тоді

$$f(n) = O\big(g(n)\big)$$

(читається як «$f$ є О велике від $g$»), якщо ∃ дійсна константа $c > 0$ і натуральна константа $n_0$, така що

$f(n) \leq cg(n)$, для $\forall\, n \geq n_0$

Ми говоримо, що $f$ зростає щонайбільше так швидко, як $g$, або не швидше, ніж $g$ (рис. 1.1).

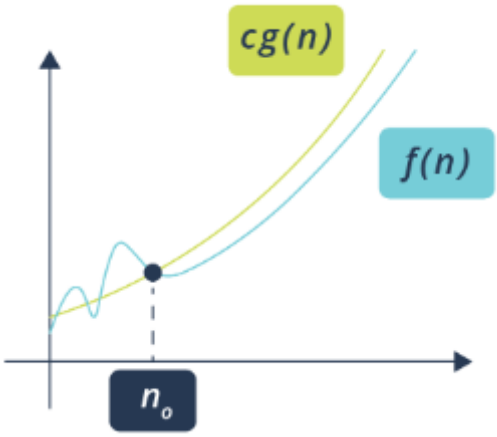


Рисунок 1.1 – До визначення $O$-нотації [41]

Мета умови $n \geq n_0$ полягає в тому, щоб ігнорувати поведінку $f$ порівняно з $g$, коли $n$ мало.

**Різниця у нотаціях**

Коли ми пишемо $f(n) = O(g(n))$, ми не використовуємо знак рівності у звичайному математичному значенні. Тут знак рівності означає «є». Іншими словами, $f(n) = O(g(n))$ є правда, натомість твердження $O\big(g(n)\big) = f(n)$ не є правильним.

На додаток до цього позначення також правильно писати $f(n) \in O(g(n))$, що означає, що $f(n)$ є членом множини функцій $O(g(n))$, або коротко «належить до О велике від $g$». Крім того, ми можемо сказати, що $f$ має порядок $g$.

Щоб побачити, як можна застосувати це визначення на практиці, розглянемо приклад.

*Зауваження*. Вибір $c$ і $n_0$ не є унікальним. Визначення вимагає існування однієї такої пари чисел. Іншими словами, щоб довести твердження типу $f(n) = O(g(n))$, ми можемо використовувати будь-яку з відповідних пар значень c і $n_0$, яких може бути багато.

**Приклад 1.1.** Розглянемо функцію $f(n) = 6n + 7$ та $g(n) = n$. Як можна довести, що $f(n) = O(g(n))$?

**Розв'язання.** Згідно з визначенням, $6n + 7 = O(n)$, якщо існує дві константи $c$ та $n_0$ такі, що

$6n + 7 \leq c \cdot n$ для $\forall\, n \geq n_0$ .

Ми можемо знайти цю константу декількома шляхами. Перший шлях наступний (рис. 1.2):

$6n + 7 \leq 6n + 7n$, де $n \geq 1$.

$= 13n$

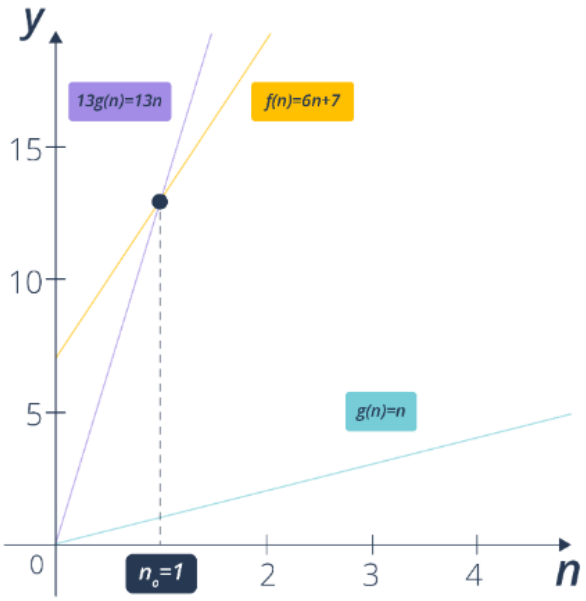


Рисунок 1.2 – До пояснення першого способу розв'язання задачі прикладу 1.1 [41]

Тому $c = 13$ і $n_0 = 1$ задовольнили визначенню, і ми довели те, що хотіли. Однак правильна і наступна логіка (рис. 1.3).

$6n + 7 \leq 6n + n$, де $n \geq 7$.

$= 7n$

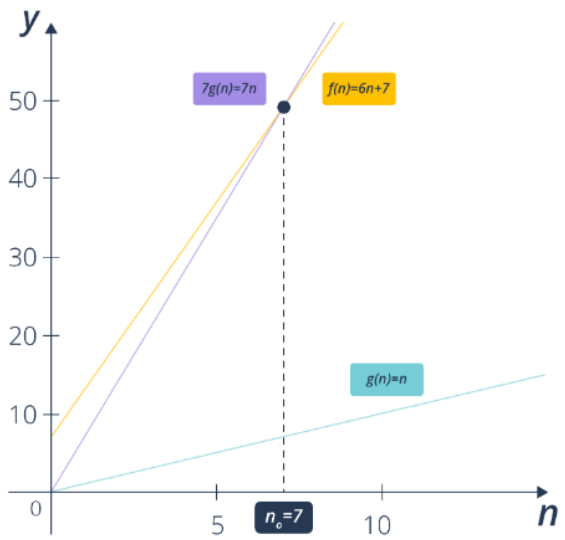


Рисунок 1.3 – До пояснення другого способу розв’язання задачі прикладу 1.1 [41]

Таким чином, $c = 7$ та $n_0 = 7$ задовольняє визначенню, що і потрібно було довести.

**Правила спрощення**

Щоб виразити швидкість зростання функції за допомогою великого $O$, можна застосувати такі правила:

*Правило 1*. Якщо функція $f$ представлена у вигляді суми кількох членів, то член, який росте швидше за інші, визначатиме порядок $f$. Іншими словами, коли аргумент прагне до нескінченності, член із найвищим темпом зростання починає домінувати, роблячи незначним внесок решти членів у темп зростання всієї функції.

*Правило 2*. Фактори, які не залежать від аргументу функції, або просто константи, можна опускати. Тобто ми не пишемо $O(4n^3)$ попри те, що це не порушує визначення. Замість цього ми пишемо $O(n^3)$.

Мета полягає у тому, щоби вираз для $g(n)$ максимально спростити.

Проілюструємо це на прикладі.

**Приклад 1.2.** Дано наступну функцію:

$$f(n) = 7n^4 + 5n^3 + 3$$

Ми хочемо знайти асимптотичний вираз для складності в $O$-нотації.

**Розв'язання.** У цьому випадку матимемо $f(n) = O(n^4)$.

Щоб переконатися, що наш результат правильний, ми можемо використати визначення, як у прикладі 1.

Згідно з визначенням, $f(n) = 7n^4 + 5n^3 + 3 = O(n^4)$ якщо існують дві константи $c$ і $n_0$ такі, що

$7n^4 + 5n^3 + 3 \leq c \cdot n^4$ для $\forall\, n \geq n_0$

Можна записати, наприклад, наступне

$7n^4 + 5n^3 + 3 \leq c \cdot n^4$ для $\forall\, n \geq 1$

$15n^4$.

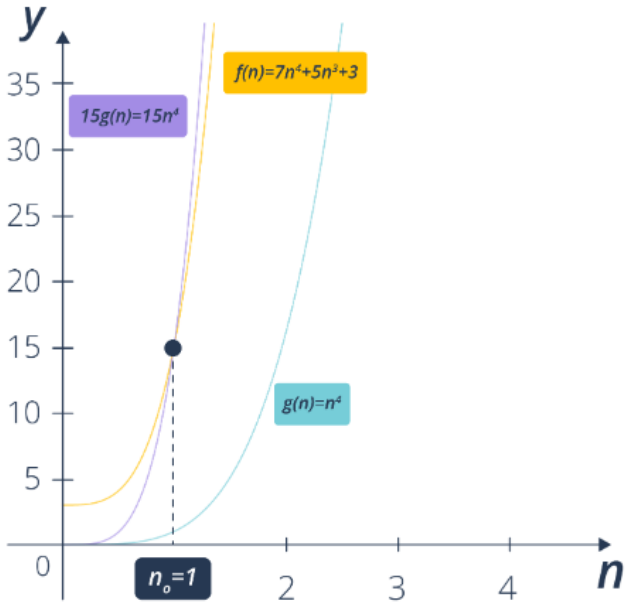


Рисунок 1.4 – До пояснення розв'язання задачі прикладу 1.2 [41]

Таким чином, $c = 15$ та $n_0 = 1$ задовольняє визначенню і, відповідно, $f(n) = O(n^4)$.

**Як вибрати функцію $g$?**

При представленні $f(n)$ у вигляді $O(g(n))$ наступні типові функції зазвичай використовуються як функції $g(n)$.

$$1 < n < \log_2(n) < \sqrt{n} < n\log_2(n) < n^2 < n^3 < \cdots < 2^n < n! < n^n$$

Цей ланцюжок нерівностей виконується не для всіх значень $n$. Він справедливий лише для $n \geq n_0$ , де $n_0$ – деяке достатньо велике число.

На наступних графіках можна спостерігати, як пов'язані перелічені вище функції. Зверніть увагу, що функції відображені на двох графіках через різницю в масштабі (рис. 1.5).

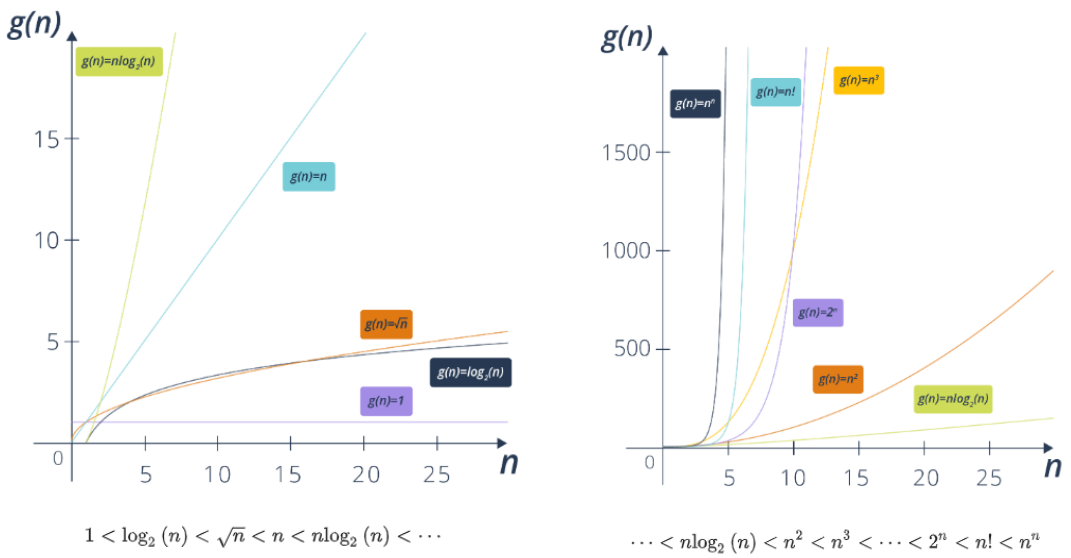


Рисунок 1.5 – Поведінка функцій $g(n)$ [41]

**Задачі для самостійного розв'язання**

Виконати індивідуальне завдання. Завдання полягає у розв'язанні двох задач, які потрібно вибрати зі списку, наведеного нижче. Правило вибору номерів наступний: $n, n+5$, де $n$ – номер студента в списку групи. У разі, якщо було досягнуто кінця списку задач, потрібно циклічно повернутися на його початок.

**Завдання**

1. Дано функцію $f(n) = 5n^2 + 1$. Знайти асимптотичну складність у $O$-нотації.

2. Дано функцію $f(n) = 7n^2$. Знайти асимптотичну складність у $O$-нотації.

3. Дано функцію $f(n) = log(n) + 2n^2 + 11$. Знайти асимптотичну складність у $O$-нотації.

4. Дано функцію $f(n) = 3n^2 + 10n - 6$. Знайти асимптотичну складність у $O$-нотації.

5. Дано функцію $fg(n) = n^2$. Знайти асимптотичну складність у $O$-нотації.

6. Довести, що $f(n) = 150n^2 + 11 = O(n^2)$.

7. Довести, що $f(n) = 3n^3 + 5 = O(n^3)$.

8. Довести, що $f(n) = 2n^2 + 10n + 3 = O(n^2)$.

9. Довести, що $f(n) = n^3 + log(n) = O(n^3)$.

10. Дано функції $f(n) = 150n^2 + 11$ та $g(n) = n^2$. Покажіть, що $f(n) = O(g(n))$.

11. Нехай f(n) = $75n^2 + 20$ і $g(n) = n^2$. Доведіть, що $f(n) = O(g(n))$.

12. Задано функції $f(n) = 200n^3 + 15$ і $g(n) = n^3$. Доведіть, що $f(n) = O(g(n))$.

13. Задано функції $f(n) = 100n^3 + 8$ та $g(n) = n^3$. Доведіть, що $f(n) = O(g(n))$.

14. Задано функції $f(n) = 50n^2 + 25$ і $g(n) = n^2$. Доведіть, що $f(n) = O(g(n))$.

15. Дано функції $f(n) = 50n^3 - 25$ та $g(n) = n^3$. Покажіть, що $f(n) = O(g(n))$.

**Контрольні питання**

1. Що таке асимптотична складність алгоритму?

2. Яким чином визначається $O$-нотація і яка її сутність?

3. Які основні правила використання $O$-нотації при аналізі алгоритмів?

4. Що означають вирази $O(1)$, $O(n)$, $O(n^2)$ в контексті асимптотичної складності?

5. Яким чином визначити асимптотичну складність алгоритму за його кодом або математичним виразом?

**Література:** [1-3, 10, 11, 41].

## Практична робота № 2

**Тема. Асимптотична складність алгоритмів. Інші нотації**

**Мета:** набути практичних навичок у розв'язанні задач на оцінку асимптотичної складності алгоритмів у $\Omega$, $\Theta$, $o$, $\theta$, $\omega$-нотаціях.

**Короткі теоретичні відомості [41]**

Велике $O$ використовується як асимптотична верхня межа функції. Поряд з верхньою межею можна визначити асимптотичну *нижню межу* функції. Для цього використовується Ω-нотація («велика омега нотація»).

**Ω-нотація**

Нехай $f$ і $g$ – дві функції, що відображають натуральні числа на додатні дійсні числа. Тоді $f(n) = \Omega(g(n))$, якщо $\exists\, c > 0$ та натуральна константа $n_0$, така, що $f(n) \geq cg(n)$ для $\forall n \geq n_0$(рис. )

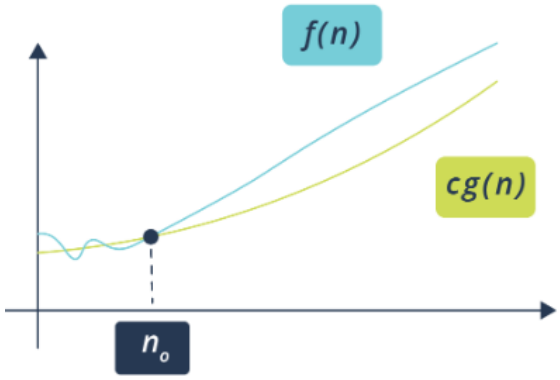


Рисунок 1.6 – До визначення Ω-нотації [41]

Ми говоримо, що $f$ росте принаймні так само швидко, як $g$.

**Приклад 1.3.** Дано функції $f$ та $g$:

$$f(n) = 3n^2 - 6n + 4$$

$$g(n) = n^2.$$

Довести, що $f(n) = \Omega(g(n))$.

**Розв'язання.** Подамо нашу функцію у наступному вигляді:

$$f(n) = n^2 + (2n^2 - 6n) + 4.$$

Видно, що $2n^2 - 6n > 0$ для $\forall n > n_0 = 3$

Таким чином, можна записати

$$3n^2 - 6n + 4 = n^2 + (2n^2 - 6n) + 4$$

$$\geq n^2 + 4$$

$$\geq 1 \cdot n^2$$

Таким чином (рис. 1.7), маємо $n_0 = 3$, $c = 1$.

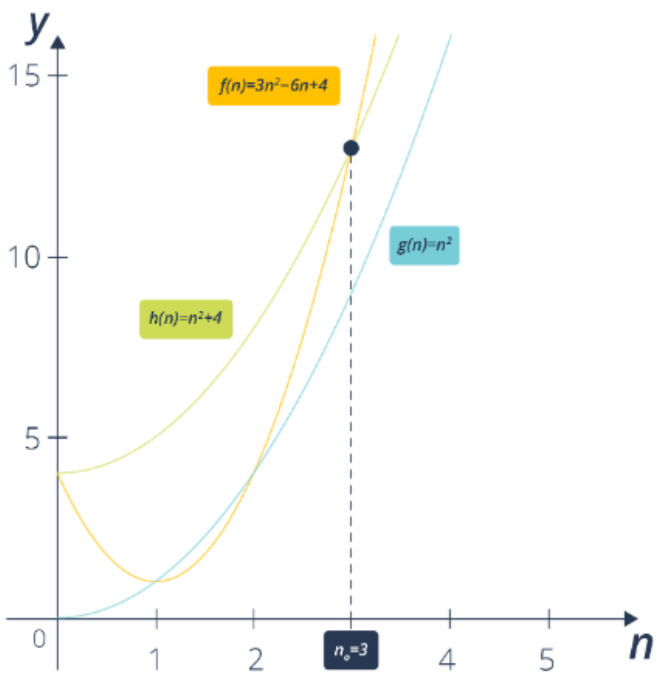


Рисунок 1.7 – До пояснення розв’язання задачі прикладу 1.3 [41]

**Θ -нотація**

Нехай $f$ та $g$ – функції натурального аргумента і дійсних значень. Тоді $f(n) = \Theta(g(n))$, якщо $\exists$ дійсні $c_1, c_2 > 0$ та натуральна константа $n_0$, така, що $c_1 g(n) \leq f(n) \leq c_2 g(n)$ для $\forall n \geq n_0$ (рис. 1.8).

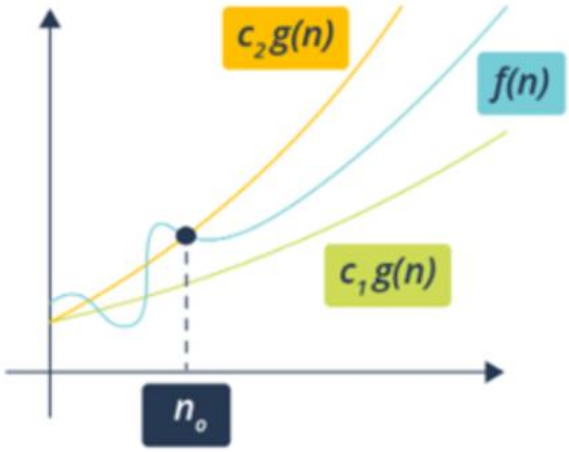


Рисунок 1.8 – До визначення Θ-нотації [41]

Важливо зауважити, що $c_1 \leq c_2$ та $n_0 = \max\{n_1, n_2\}$ і є константами $O()$ та $\Omega()$ відповідно.

Еквівалентно, $f(n) = \Theta(g(n))$, якщо $f(n)$ є одночасно $O\big(g(n)\big)$ і $\Omega(g(n))$.

Таким чином, для доведення того, що $f(n) = \Theta(g(n))$, необхідно роздільно довести, що $f(n) = O\big(g(n)\big)$ і $f(n) = \Omega(g(n))$. Ці питання розглянуті вище у цьому і попередньому занятті.

**Малі асимптотичні нотації**

*Визначення.* Нехай $f$ і $g$ – дві функції, що відображають натуральні у простір дійсних чисел. Тоді

$$f(n) = o\big(g(n)\big)$$

(читається як «$f$ є $o$ маленьке від $g$»), якщо ∃ дійсна константа $c > 0$ і натуральна константа $n_0$, така що $f(n) < cg(n)$, для $\forall\, n \geq n_0$. Ми говоримо, що $f$ зростає повільніше, ніж $g$.

Відмінність між $O$ та $o$ нотаціями полягає у тому, що останнє є більш строгим:

$f(n) = o\big(g(n)\big) \Rightarrow f(n) = O(g(n))$. Зворотне твердження не є вірним.

*Визначення*. Нехай $f$ і $g$ – дві функції, що відображають натуральні числа на додатні дійсні числа. Тоді $f(n) = \omega(g(n))$, якщо $\exists\, c > 0$ та натуральна константа $n_0$, така, що $f(n) > cg(n)$ для $\forall n \geq n_0$.

Відмінність між $\Omega$ та $\omega$ нотаціями полягає у тому, що останнє є більш строгим:

$f(n) = \omega\big(g(n)\big) \Rightarrow f(n) = \Omega(g(n))$. Зворотне твердження не є вірним.

**Асимптотичні нотації та межі.**

Методи по визначенню констант $c$ та $n_0$, розглянуті вище, не єдині. Розглянемо альтернативні підходи на основі меж.

**Метод меж**

Припустимо, що є дві функції $f$ та $g$, тоді для порівняння порядку зростання цих функцій потрібно визначити межу $\lim\limits_{n\to\infty} \frac{f(n)}{g(n)}$.

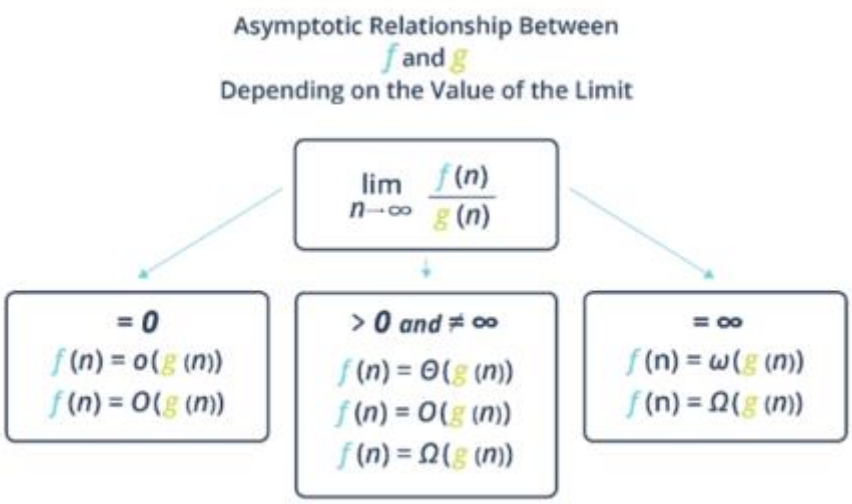


Рисунок 1.9 – Асимптотичне відношення між $f$ та $g$, що залежить від значення межі [41]

**Приклад 1.4.** Дано функції

$$f(n) = 2\log_2(n)$$

$$g(n) = \sqrt{n}.$$

Показати, що $2\log_2(n) = O(\sqrt{n})$.

**Розв'язання.** Для того, щоби показати, що $2\log_2(n) = O(\sqrt{n})$, нам потрібно перевірити, чи дорівнює значення наступної межі 0, або деякій константі $c > 0$:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{2\log_2(n)}{\sqrt{n}} = \frac{\infty}{\infty}.$$

Для розв'язання невизначеності $\frac{\infty}{\infty}$ застосуємо правило Лопіталя:

$\left(\log_a(n)\right)' = \frac{1}{nln(a)}$,

$\left(\sqrt{n}\right)' = \frac{1}{2\sqrt{n}}$,

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{2\log_2(n)}{\sqrt{n}} = \lim_{n\to\infty} \frac{(2\log_2(n))'}{(\sqrt{n})'} = \frac{4}{\ln(2)} \lim_{n\to\infty} \frac{1}{\sqrt{n}} = 0.$$

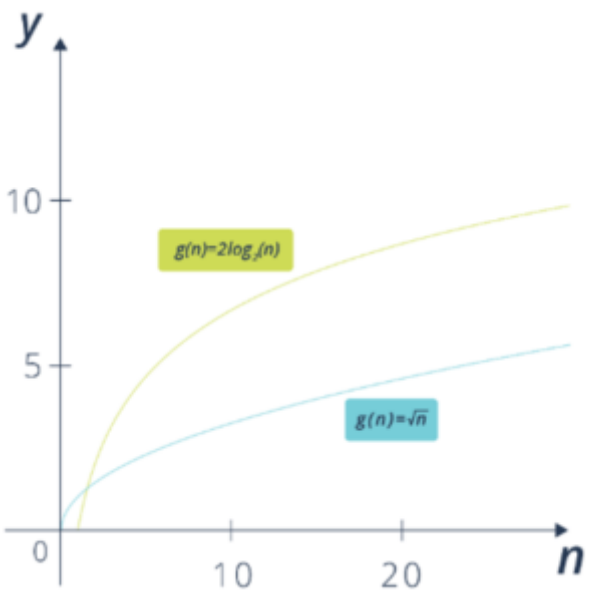


Рисунок 1.7 – До пояснення розв’язання задачі прикладу 1.4 [41]

**Задачі для самостійного розв’язання**

Виконати індивідуальне завдання. Завдання полягає у розв’язанні двох задач, які потрібно вибрати зі списку, наведеного нижче. Правило вибору номерів наступний: $n, n+5$, де $n$ – номер студента в списку групи. У разі, якщо було досягнуто кінця списку задач, потрібно циклічно повернутися на його початок.

**Завдання**

1. Маємо дві функції $f(n) = 4n^3 - 12n + 9, g(n) = n^3$. Довести, що $f(n) = \Omega(g(n))$.

2. Маємо дві функції $f(n) = 4n^3 - 12n + 9$ і $g(n) = n^3$. Довести, що $f(n) = \Omega(g(n))$.

3. Задано функції $f(n) = 3n^3 - 8n^2 + 15$ та $g(n) = n^3$. Покажіть, що $f(n) = \Omega(g(n))$.

4. Дано функції $f(n) = 2n^3 + 7n^2 - 4$ і $g(n) = n^3$. Довести, що $f(n) = \Omega(g(n))$.

5. Розглянемо функції $f(n) = 5n^3 - 10n + 20$ та $g(n) = n^3$. Покажіть, що $f(n) = \Omega(g(n))$.

6. Нехай $f(n) = 6n^3 - 9n^2 + 12n$ і $g(n) = n^3$. Довести, що $f(n) = \Omega(g(n))$.

1. Дано функції $f(n) = n^2 + 2n + 8$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

7. Задано функції $f(n) = n^3 + 2n^2 - 5n + 8$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

8. Маємо функції $f(n) = n^4 + 2n^3 - 5n^2 + 8$ та $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

9. Розглянемо функції $f(n) = n^4 + 2n^2 - 5n + 8$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

10. Нехай $f(n) = n^4 + 2n^3 - 5n + 8$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

11. Дано функції $f(n) = n^3 + 2n^2 - 5n + 8$, $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

12. Дано функції $f(n) = n^2 + 3n + 10$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

13. Задано функції $f(n) = n^3 - n^2 + 2$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

14. Маємо функції $f(n) = n^4 - 2n^3 + 3n + 7$ та $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

15. Розглянемо функції $f(n) = n^4 - 3n^2 + 4n + 9$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

**Контрольні питання**

1. Що таке асимптотична складність алгоритму?

2. Які інші нотації, крім О-нотації, використовуються для вираження асимптотичної складності?

3. Як визначити асимптотичну складність алгоритму за допомогою символів Θ і Ω?

4. Яка різниця між О-нотацією, Θ-нотацією і Ω-нотацією?

5. Які основні властивості інших нотацій, таких як o (маленька o), ω (маленька омега) та $o$ (маленька o з верхнім індексом)?

**Література:** [1-3, 10, 11, 41].

## Практична робота № 3

**Тема. Алгоритми сортування та їх складність. Порівняння алгоритмів сортування**

**Мета:** опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

### Короткі теоретичні відомості

**Постановка задачі сортування**

Сортування є одним із фундаментальних завдань у області обробки даних та алгоритмів. Його мета полягає в упорядкуванні набору елементів у відповідності до певного критерію. Під критерієм може розумітися зростання або спадання значень, лексикографічне або числове порівняння, а також інші параметри відповідно до специфіки задачі. Розглянемо деякі з основних алгоритмів сортування масивів і технологію оцінки та аналізу їх асимптотичної складності.

**Перелік існуючих алгоритмів**

На сьогоднішній день існує велика кількість різних алгоритмів сортування, які можуть бути використані для розв’язання різноманітних завдань. Серед найпоширеніших можна виділити такі:

1. *Сортування вибором (Selection Sort)*. Цей алгоритм вибирає найменший елемент зі списку і переміщує його на початок. Потім він повторює цей процес для решти списку.

2. *Сортування бульбашкою (Bubble Sort)*. Простий алгоритм, який порівнює сусідні елементи і переміщує найбільший елемент у кінець списку. Повторює цей процес для всіх елементів до тих пір, поки список не буде відсортований.

3. *Сортування вставлянням (Insertion Sort)*. Алгоритм, який збільшує відсортовану частину списку, додаючи кожен наступний елемент на правильне місце.

4. *Сортування злиттям (Merge Sort)*. Рекурсивний алгоритм, який розбиває список навпіл, сортує кожну половину, а потім об'єднує їх впорядкованим чином.

5. *Швидке сортування (Quick Sort)*. Ефективний алгоритм, який вибирає опорний елемент, розбиває список на дві підсписки, менший і більший за опорний елемент, і потім рекурсивно сортує кожен з них.

Це лише декілька з найбільш відомих алгоритмів сортування. Кожен з них має свої переваги і недоліки, а вибір конкретного алгоритму залежить від вхідних даних, обсягу списку, а також вимог до швидкодії та пам’яті.

**Алгоритм сортування вибором**

Розглянемо найпростіший алгоритм сортування вибором, оцінимо його складність у найгіршому та найкращому випадку, та асимптотичну складність окремих його процедур.

Алгоритм сортування вибором полягає в наступному (рис. 1.):

Для кожного елемента починаючи з першого до передостаннього:

1) Знайти індекс найменшого елемента в залишку списку.

2) Поміняти поточний елемент з найменшим елементом.

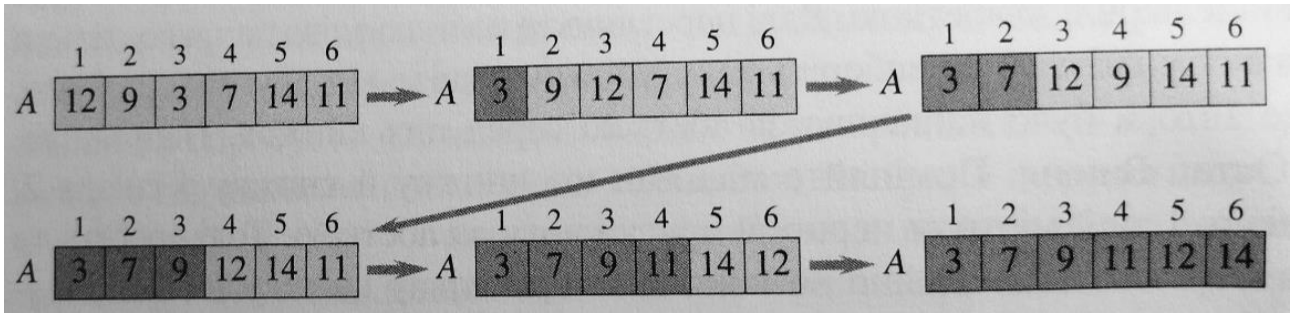


Рисунок 1.8 – Алгоритм сортування вибором [1]

**Оцінка складності**

*Найгірший випадок:* $O(n^2)$ – коли список відсортований у зворотному порядку або майже відсортований. У цьому випадку алгоритм все ще перевіряє всі можливі пари елементів.

У найкращому випадку алгоритм сортування вибором все одно має складність $O(n^2)$, оскільки навіть якщо список вже відсортований, алгоритм все одно потребує порівнянь між всіма елементами, щоб знайти найменший елемент на кожній ітерації.

Давайте розглянемо це більш детально. У найкращому випадку, коли список вже відсортований, на кожній ітерації вибирається наступний мінімальний елемент за допомогою вкладених циклів. Кожна ітерація внутрішнього циклу потребує порівняння $n - i$ елементів (де $n$ – розмір списку, а $i$ – номер поточного елемента). Отже, сумарна кількість порівнянь у найкращому випадку є сумою арифметичної прогресії і складає $n + (n-1) + (n-2) + \ldots + 1) = n(n+1)/2$, що відповідає $O(\frac{n^2}{2})$, що в асимптотичній нотації є $O(n^2)$.

Таким чином, навіть у найкращому випадку алгоритм сортування вибором має квадратичну складність $O(n^2)$.

Складність процедури переставляння двох елементів у списку становить $O(1)$, оскільки це просто операція обміну двох значень.

**Алгоритм сортування вставлянням**

1. Проходимо по кожному елементу списку, починаючи з другого (рис. 1.9).
2. На кожній ітерації вибираємо поточний елемент і порівнюємо його з елементами, що знаходяться перед ним у відсортованій частині списку.
3. Якщо знаходимо елемент, менший за поточний, переміщуємо його на одну позицію вправо.
4. Вставляємо поточний елемент на звільнену позицію.

**Оцінка складності**

Найкращий випадок: $O(n)$ – коли вихідний список вже відсортований, і кожен новий елемент вставляється безпосередньо за останнім елементом.

Найгірший випадок: $O(n^2)$ – коли вихідний список відсортований у зворотному порядку, і для кожного елементу потрібно перебрати усі попередні елементи, щоб вставити його на потрібне місце.

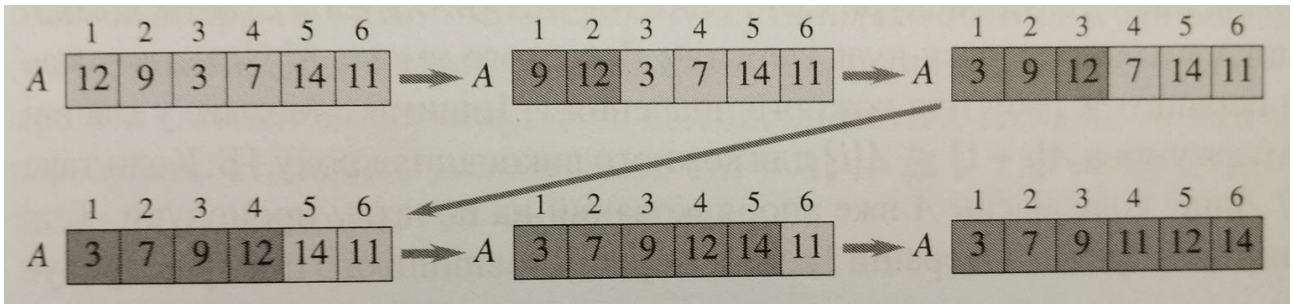


Рисунок 1.9 – Алгоритм сортування вставлянням [1]

Для простоти аналізу асимптотики напишемо код на Python:

```
import time
# Сорутвання методом вставляння (включення)

def insertion_sort(nums):
    # починаємо з другого елемента
    for j in range(1, len(nums)):
        key = nums[j]
        # Зберігаємо посилання на індекс першого елемента
        i = j - 1
        # Елементи відсортованого сегрмента переміщуємо вперед, якщо вони більше елемента вставки
        while i >= 0 and nums[i] > key:
            nums[i + 1] = nums[i]
            i -= 1
        # Вставляємо елемент
        nums[i + 1] = key
```

**Основна теорема рекурсії. Стратегія «розділяй і володарюй»**

Часто ефективними алгоритмами, зокрема сортування, є алгоритми, в основі яких лежить стратегія «розділяй і володарюй». Суть стратегії полягає у розбитті задачі на підзадачі з послідуючою консолідацією результату. Реалізація цієї стратегії призводить до рекурсії. Як визначити, який порядок складності у такому випадку?

Одна з можливих відповідей на це питання – застосування основної теореми рекурсії.

Нехай:

– $a$ – кількість підзадач;

– розмір кожної підзадачі зменшується в $b$ разів і стає рівним $[\frac{n}{b}]$\$

– складність консолідаціїї нехай є $O(n^d)$.

– час роботи такого алгоритму, виражений рекурентно, буде $T(n) = aT([\frac{n}{b}]) + O(n^d)$.

*Теорема*. Нехай $T(n) = aT([\frac{n}{b}]) + O(n^d)$ для деяких $a > 0, b > 1, d \geq 0$, тоді

$$F_n = \begin{cases} O(n^d), & \text{якщо } d > log_b a, \\ O(n^d logn), & \text{якщо } d = log_b a, \\ O(n^{log_b a}), & \text{якщо } d < log_b a. \end{cases}$$

**Алгоритм сортування зливанням**

Це приклад алгоритму сортування, побудований на стратегії «розділяй і володарюй».

Парадигма «розділяй і володарюй» на кожному рівні рекурсії містить три кроки:

– *Поділ* задачі на декілька підзадач.

– *Володарювання* над підзадачами шляхом їх рекурсивного розв'язку.

– *Комбінування* розв'язків підзадач у розв'язок початкової задачі.

Алгоритм *сортування зливанням (merge sort)* точно слідує парадигмі "розділяй і володарюй":

– *Поділ.* Ділимо $n$-елементну послідовність на дві підпослідовності по $\frac{n}{2}$ елементів.

– *Володарювання.* Рекурсивно сортуємо ці дві послідовності методом злиття.

– *Комбінування.* Поєднуємо дві відсортовані підпослідовності для отримання остаточного відсортованого результату.

Код на Python:

```
def merge(left, right):

    #Зливає два відсортованих масиви left і right у один

    result = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result += left[i:]
    result += right[j:]
    return result

def mergesort(list):

    #Рекурсивна функція сортування з використанням попередньої функції

    if len(list) < 2:
        return list
    middle = len(list) // 2
    left = mergesort(list[:middle])
    right = mergesort(list[middle:])
                        return merge(left, right)
```

**Оцінка складності**

Найкращий випадок: $O(nlog\,n)$ – коли розмір списку поділяється на рівні підсписки, і кожен рівень рекурсії має складність $O(n)$.

Найгірший випадок: $O(nlog\,n)$ – аналогічно найкращому випадку, оскільки кожен рівень рекурсії все одно має складність $O(n)$.

**Порівняльний аналіз методів сортування**

У табл. 1.1 підсумовані властивості чотирьох розглянутим нами алгоритмів сортування [1].

Таблиця 1.1 – Характеристики асимптотичної складності різних алгоритмів сортування

| Алгоритм | Час роботи в найгіршому разі | Час роботи в найліпшому разі | Кількість переставлянь | На місці |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Сортування вибором | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n)$ | так |
| Сортування вставлянням | $\Theta(n^2)$ | $\Theta(n)$ | $\Theta(n^2)$ | так |
| Сортування зливанням | $\Theta(nlog\ n)$ | $\Theta(nlog\ n)$ | $\Theta(nlog\ n)$ | ні |
| Швидке сортування | $\Theta(n^2)$ | $\Theta(nlog\ n)$ | $\Theta(n^2)$ | так |

Таким чином, можна зробити декілька важливих зауважень. У той час, як найбільш ефективним можна назвати алгоритм зливання, сортування вибором є найгіршим алгоритмом у порівнянні з усіма іншими, так як на всіх рівнях має квадратичну асимптотику $\Theta(n^2)$. Однак має найкращу ефективність з точки зору складності переставляння: $\Theta(n)$. Якщо переміщення елементів масиву вимагає особливо багато часу – можливо через те, що вони великі і зберігаються на повільному носії на кшталт диска – то використання сортування вибором може бути доречним навіть, ніж алгоритм зливання, якій до того ж переставляння робить не на місці.

**Задачі для самостійного розв'язання**

Виконати індивідуальне завдання. Завдання полягає у розв'язанні всіх задач, наведених нижче. Тобто студенти виконують *єдиний варіант для всіх*.

**Завдання**

1. Вивчити самостійно і записати (будь-яким способом) алгоритм бульбашкового сортування. Оцінити асимптотику алгоритму сортування методом бульбашки в найгіршому і в найкращому випадку. Порівняти за цими показниками бульбашковий алгоритм з алгоритмом сортування вставлянням. Чому на практиці бульбашковий алгоритм виявляється менш ефективним у порівнянні з сортуванням методом зливанням?

2. Оцінити асимптотичну складність алгоритму сортування зливанням, скориставшись основною теоремою рекурсії.

3. Вивчити і записати (будь-яким способом) самостійно алгоритм швидкого сортування. Оцінити асимптотичну складність алгоритму швидкого сортування, скориставшись основною теоремою рекурсії.

## Контрольні питання

1. Що таке асимптотична складність алгоритму сортування і чому вона важлива для порівняння алгоритмів?

2. Які алгоритми сортування мають квадратичну складність у найгіршому випадку? Поясніть, чому це може бути проблемою для великих обсягів даних.

3. В чому полягає перевага сортування злиттям над сортуванням вставками для великих наборів даних?

4. Які алгоритми сортування використовуються для сортування списків у стандартних бібліотеках мов програмування, таких як Python, Java або C++?

5. Яка різниця між алгоритмами сортування злиттям і швидким сортуванням? У яких випадках краще використовувати кожен з цих алгоритмів?

6. Які фактори слід враховувати при виборі алгоритму сортування для конкретної задачі?

**Література:** [1, с. 27-30, с. 42-49, с. 52-61, 2, 3, 10, 11, 41].

## Практична робота № 4

**Тема. Алгоритми пошуку та їх складність**

**Мета:** опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

## Короткі теоретичні відомості

**Задача пошуку**

Задача пошуку полягає в знаходженні певного елемента в контейнері, який містить $n$ елементів. Якщо такий елемент буде знайдено – повернути його індекс.

**Лінійний пошук**

Одним з найпростіших алгоритмів пошуку є *лінійний пошук*. Сенс алгоритму полягає в тому, що пошук починають з першого елементу масиву.

Якщо поточний елемент списку не дорівнює шуканому значенню, то здійснюється перехід до наступного елементу. Таким чином, у результаті кожної перевірки область пошуку зменшується на один елемент.

Лінійний пошук також відомий, як *метод перебору* або «грубої сили». Працює за час $O(n)$, де $n$ – довжина списку на вході. Лінійна асимптотика є зрозумілою, виходячи з простоти самого алгоритму.

Код на Python:

```
# Якщо елемент не буде знайдено, виводиться -1
def linear_search(a_list, x):
    i, length = 0, len(a_list)
    while i < length and x != a_list[i]:
        i += 1
         return i if i < length else -1
```

Чи можна більшої ефективності, ніж $O(n)$? – Можна. Якщо попередньо масив впорядкувати.

**Пошук у відсортованому масиві. Бінарний пошук**

Це класичний алгоритм пошуку елемента у відсортованому списку, який використовує дроблення масиву на половини. Алгоритм можна реалізувати рекурсивно. Нижче наведено реалізацію для відсортованих списків без повторюваних елементів. Складність алгоритму – $O(log(n))$, де $n$ – довжина списку на вході.

*Алгоритм полягає у наступному.*

1. Знаходиться середній елемент послідовності. Для цього перший і останній індекси зв'язуються зі змінними, а індекс середнього елемента обчислюється.

2. Значення середнього елемента порівнюється з шуканим значенням. Залежно від того, більше воно чи менше значення середнього елемента, подальший пошук буде відбуватися тільки в лівій або тільки в правій половинах масиву. Якщо значення середнього елемента виявляється рівним шуканому, пошук завершується.

3. Інакше одна з меж досліджуваної послідовності зсувається. Якщо шукане значення більше значення середнього елемента, то нижня межа

зсувається за середній елемент на один елемент праворуч. Якщо шукане значення менше значення середнього елемента, то верхня межа зсувається на елемент перед середнім.

4. Знову знаходиться середній елемент тепер уже в обраній половині. Описаний вище алгоритм повторюється для даного зрізу.

Код на Python:

```
def bin_search(a_list, x, left, right):
    if left > right or len(a_list) == 0:
        return -1
    middle = (left + right) // 2
    if a_list[middle] == x:
        return middle
    elif (a_list[middle] < x):
        return bin_search(a_list, x, middle + 1, right)
    else:       # a_list[middle] > x
            return bin_search(a_list, x, left, middle - 1)
```

Алгоритм побудований за відомою нам з попередньої практичної роботи стратегією «розділяй і володарюй». Його асимптотична складність складає $O(\log n)$, і легко може бути знайдена за допомогою основної теореми рекурсії.

**Задачі для самостійного розв'язання**

Виконати індивідуальне завдання. Завдання полягає у розв'язку всіх задач, наведених нижче. Тобто студенти виконують *єдиний варіант для всіх*.

**Завдання**

1. Оцінити асимптотичну складність алгоритму лінійного пошуку у $O$-нотації в найгіршому і в найкращому випадку. Як можна покращити алгоритм лінійного пошуку?

2. Оцінити асимптотичну складність алгоритму бінарного пошуку у $O$-нотації в найгіршому і в найкращому випадку.

3. Побудувати алгоритм тернарного пошуку і оцінити його асимптотичну складність алгоритму у $O$-нотації в найгіршому і в найкращому випадку. Який з алгоритмів є оптимальнішим: бінарний, чи тернарний? Обґрунтувати відповідь відповідними обчисленнями.

4. Порівняти ефективність алгоритмів лінійного, бінарного та тернарного пошуку для різних розмірів вхідного списку. Для цього провести

експериментальне дослідження та побудувати графіки залежності часу виконання алгоритму від розміру вхідного списку.

5. Порівняти алгоритми пошуку за їхньою здатністю працювати з відсортованими та не відсортованими списками. Провести аналіз впливу відсортованості списку на час виконання кожного алгоритму.

6. Розглянути сценарії використання кожного з алгоритмів пошуку у практичних задачах і обґрунтувати вибір кожного алгоритму в конкретному випадку.

**Контрольні питання**

1. Що таке алгоритм пошуку і чому він важливий у контексті комп'ютерних наук?

2. Які основні критерії оцінки ефективності алгоритмів пошуку?

3. Що таке лінійний пошук, і як він працює?

4. Які умови повинні бути виконані для успішного застосування бінарного пошуку?

5. Які переваги та недоліки використання бінарного пошуку порівняно з іншими алгоритмами пошуку?

6. Що таке тернарний пошук, і в чому його відмінність від бінарного пошуку?

**Література:** [1, с. 9-20, с. 23-27, 2, 3, 10, 11, 41].

# 3 КРИТЕРІЇ ОЦІНЮВАННЯ ЗНАНЬ СТУДЕНТІВ

Практичні роботи студенти виконують у рамках першого змістового модуля «Алгоритми і структури даних».

Всього студент має виконати дев'ять практичних завдань. За виконання всіх індивідуальних завдань в рамках кожної практичної роботи студент може отримати максимально 27 балів. Максимальна кількість балів, яку можна отримати студенти за виконання однієї лабораторної роботи, складає 3 балів: 1 – «задовільно», 2 – «добре», 3 – «відмінно».

За кожне індивідуальне завдання в рамках практичної роботи виставляють:

– «відмінно», якщо студент виконав лабораторну роботу, написав звіт і захистив його, тобто правильно та вичерпно відповів на всі питання. Допускаються незначні неточності під час відповіді на одне з контрольних питань;

– «добре», якщо студент виконав лабораторну роботу, написав звіт і захистив його, але мав незначні помилки;

– «задовільно», якщо виконав лабораторну роботу, написав звіт і захистив його, але під час захисту не відповів на декілька питань і мав незначні помилки під час оформлення звіту.

Розподіл балів, що отримують студенти, за видами занять наведено у табл. 3.1 розділу 3.

Таблиця 3.1 – Розподіл балів за видами занять

| **Критерії оцінювання** | | |
|---|---|---|
| Вид роботи | Зміст | Бали |
| Робота на лекціях | Активна участь у дискусіях, розгляд практичних кейсів тощо | 10 |
| Завдання з ЛР (виконання, захист) | ЛР 1 Налаштування Git-GitHub середовища | 1 |
| | ЛР 2 Налаштування та робота в середовищі Anaconda+Quarto | 2 |

| | | |
|---|---|---|
| | ЛР 3 Аналіз складності алгоритмів. Алгоритми сортування | 5 |
| | ЛР 4 Рекурсія. Стратегія «розділяй і володарюй» | 5 |
| | ЛР 5 Алгоритми пошуку | 5 |
| | ЛР 6 Структури даних стек і черга | 5 |
| | ЛР 7 Структури даних купа, дерево, геш-таблиця | 5 |
| | ЛР 8 Структура даних граф. Алгоритми на графах | 5 |
| | ЛР 9 Алгоритми на рядках | 5 |
| | ЛР10 Стиснення даних. Жадібна стратегія на прикладі кодування Гафмена | 5 |
| Завдання з ПР (виконання, захист) | ПР 1 Асимптотична складність алгоритмів. O-нотація | 3 |
| | ПР 2 Асимптотична складність алгоритмів. Інші нотації | 3 |
| | ПР 3 Алгоритми сортування та їх складність. Порівняння алгоритмів сортування | 3 |
| | ПР 4 Алгоритми пошуку та їх складність | 3 |
| | ПР 5 Графи. Ациклічні графи | 3 |
| | ПР 6 Графи. Найкоротші шляхи. Дерева | 3 |
| | ПР 7 Алгоритми на рядках | 3 |
| | ПР 8 Жадібні алгоритми. Наближений розв’язок екстремальних задач | 3 |
| | ПР 9 Стиснення даних. Кодування і декодування Гафмена | 3 |
| Контроль | Іспит (тестування) | 20 |
| | **Усього балів** | **100** |

## СПИСОК ЛІТЕРАТУРИ


1. Кормен Томас Г. Алгоритми доступно / пер. з англ. Кирила Яценка. Київ: К.І.С, 2021. 194 с.

2. Кормен Томас Г., Лейзерсон Чарлз Е., Рівест Роналд Л., Стайн Кліфорд. Вступ до алгоритмів. Київ: К.І.С, 2023. 1288 с.

3. Dasgupta S. et al. (2006). *Algorithms* (1st ed.). McGraw-Hill Education. ISBN-10: 9780073523408, ISBN-13: 978-0073523408.

4. Aho A. V., Ullman J. D. & Hopcroft J. E. (n.d.). *Data Structures and Algorithms*. Addison-Wesley Professional, 2000. 428 p. URL: https://doc.lagout.org/Alfred%20V.%20Aho%20-%20Data%20Structures%20and%20Algorithms.pdf (дата звернення 20.01.2023).

5. Sydorenko, V., Perekrest, A., Shendryk, V., Shendryk, S. (2023). Machine Learning Optimization of Air Heating Time in the Heating Control System of a Smart House. In International Conference «New Technologies, Development and Application». Lecture Notes in Networks and Systems, vol 707. Springer, Cham. pp. 36-44. https://doi.org/10.1007/978-3-031-34721-4_5

6. Guchenko M., Sydorenko V., Belska V., Liutenko M., Fesenko N. DComFra Project Learning Module M20 Advanced Spreadsheets in Mathematical Modeling Tasks of Electrical and Computer Engineers Education. Proceedings of the 20th IEEE International Conference on Modern Electrical and Energy Systems, MEES 2021.

7. Wirth N. Algorithms and Data Structures. Prentice Hall, 1985, 288 p. URL: https://people.inf.ethz.ch/wirth/AD.pdf (дата звернення 20.01.2023).

8. Knuth D. E. The Art of Computer Programming, Volume 3: Sorting and Searching. (2nd ed.). Addison-Wesley Professional, 1998.

9. Бхаргава А. Грокаємо алгоритми: Ілюстрований посібник для програмістів і допитливих / Адітья Бхаргава. Пер. із англ. Олександра Медведа. Київ: ArtHuss, 2023. 288 с.

10. Шаховська Н. Б., Голощук Р. О. Алгоритми і структури даних: посібник. Львів: Магнолія–2006, 2022. 215 с.

11. Креневич А. П. Алгоритми і структури даних: підручник. Київ: ВПЦ «Київський Університет», 2021. 200 с. URL: https://www.mechmat.univ.kiev.ua/wp-content/uploads/2021/09/pidruchnyk-alhorytmy-i-struktury-danykh.pdf

**Інформаційні ресурси**

14. git. URL: https://git-scm.com/downloads (дата звернення: 14.07.2023).

15. git (укр.). URL: https://git-scm.com/book/uk/v2 (дата звернення: 14.07.2023).

16. Teacher's DevOps Course (Lecture 10). URL: https://www.youtube.com/watch?v=Md8RW6tKCNg (дата звернення: 14.07.2023).

17. Anaconda (Python distribution). Електронний ресурс. Точка доступу: https://uk.wikipedia.org/wiki/Anaconda_(Python_distribution)

18. Install Python 3.8 on Windows 10 - 4 of 10 - Create and Activate a Virtual Environment with Pipenv. Електронний ресурс. Точка доступу: https://www.youtube.com/watch?v=l2_5wiYg5dY

19. Conda. Електронний ресурс. Точка доступу: https://conda.io/en/latest/

20. Welcome to Quarto®. Електронний ресурс. URL: https://quarto.org/ (дата звернення: 14.07.2023).

21. Markdown Basics. Електронний ресурс. URL: https://quarto.org/docs/authoring/markdown-basics.html (дата звернення: 14.07.2023).

22. Callout Blocks. Markdown Syntax. URL: https://quarto.org/docs/authoring/callouts.html (дата звернення: 14.07.2023).

23. Literate prigramming (Вікіпедія). URL: https://en.wikipedia.org/wiki/Literate_programming (дата звернення: 14.07.2023).

24. Posit. (n.d.). Put data into production with Posit Connect. URL: https://posit.co/ (дата звернення: 14.07.2023)

25. Markdown. URL: https://uk.wikipedia.org/wiki/Markdown (дата звернення: 14.07.2023)

26. LaTeX. URL: https://uk.wikipedia.org/wiki/LaTeX (дата звернення: 14.07.2023)

27. sandino. 2013. “Cheat Sheet of Markdown.” Article. https://github.com/sandino/Markdown-Cheatsheet#emphasis.

28. Python Tutorial – Python Full Course for Beginners. Електронний ресурс. URL: https://www.youtube.com/watch?v=_uQrJ0TkZlc (дата звернення: 14.07.2023)

29. Matplotlib: Visualization with Python. Електронний ресурс. URL: https://matplotlib.org/ (дата звернення: 14.07.2023)

30. Stack Implementation in Python. Електронний ресурс. URL: https://codereview.stackexchange.com/questions/82802/stack-implementation-in-python (дата звернення: 14.07.2023)

31. Python: модуль Queue. Електронний ресурс. URL: https://john16blog.blogspot.com/2012/05/python-queue.html (дата звернення: 14.07.2023)

32. Граф (абстрактний тип даних). Електронний ресурс. URL: https://uk.wikipedia.org/wiki/%D0%93%D1%80%D0%B0%D1%84_(%D0%B0%D0%B1%D1%81%D1%82%D1%80%D0%B0%D0%BA%D1%82%D0%BD%D0%B8%D0%B9_%D1%82%D0%B8%D0%BF_%D0%B4%D0%B0%D0%BD%D0%B8%D1%85) (дата звернення: 14.07.2023)

33. Graph Theory and Graphs in Python. Електронний ресурс. URL: https://python-course.eu/applications-python/graphs-python.php (дата звернення: 14.07.2023)

34. NetworkX. Network Analysis in Python. URL: https://networkx.org/ (дата звернення: 14.07.2023)

35. *Steven Skiena (англ.)* The Algorithm Design Manual (2nd ed.). — 2010. — с. 152 of section 5.2: Data Structures for Graphs. — ISBN 0-387-00163-8.

36. Черга з пріоритетом. Електронний ресурс. URL: https://uk.wikipedia.org/wiki/%D0%A7%D0%B5%D1%80%D0%B3%D0%B0_%D0%B7_%D0%BF%D1%80%D1%96%D0%BE%D1%80%D0%B8%D1%82%D0%

B5%D1%82%D0%BE%D0%BC#:~:text=priority%20queue (дата звернення: 14.07.2023)

37. Heap queue (or heapq) in Python. Електронний ресурс. URL: https://www.geeksforgeeks.org/heap-queue-or-heapq-in-python/ (дата звернення: 14.07.2023)

38. heapq — Heap queue algorithm. Електронний ресурс. URL: https://docs.python.org/3/library/heapq.html (дата звернення: 14.07.2023)

39. Huffman Coding | Greedy Algo-3. Електронний ресурс. URL: https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/ (дата звернення: 14.07.2023)

40. Huffman Decoding. Електронний ресурс. URL: https://www.geeksforgeeks.org/huffman-decoding/?ref=gcse (дата звернення: 14.07.2023)

41. Computer Science for Switchers від EPAM University. Електронний ресурс. URL: https://learn.epam.com , (дата звернення: 2022)

42. Algorithms. Khan Academy. Електронний ресурс. URL: https://www.khanacademy.org/computing/computer-science/algorithms (дата звернення 20.01.2023).

43. Курс CS50. Електронний ресурс. URL: https://prometheus.org.ua/course/course-v1:Prometheus+CS50+2019_T1 (дата звернення 20.01.2023).

44. Розробка та аналіз алгоритмів. URL: https://apps.prometheus.org.ua/learning/course/KPI/Algorithms101/2015_Spring/home (дата звернення 20.01.2023).

Додаток А

**Зразок оформлення титульної сторінки звіту**

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО

Кафедра комп’ютерної інженерії та електроніки

ЗВІТ З ПРАКТИЧНИХ РОБІТ
з навчальної дисципліни
«Алгоритми та методи обчислень»

Тема «_____»

Студент гр. _____ ПІБ _____
Викладач _____ ПІБ _____

Кременчук 20__

Методичні вказівки щодо виконання практичних і самостійної робіт з навчальної дисципліни «Алгоритми та методи обчислень» для студентів денної форми навчання зі спеціальності 122 – «Комп'ютерні науки» освітньо-професійної програми «Комп'ютерні науки» освітнього ступеня «Бакалавр» (частина 1)

Укладач к. т. н., доц. В. М. Сидоренко
Відповідальний за випуск к. т. н., доц. Н. М. Істоміна

Підп. до др._____Формат 60х84 1/16. Папір тип. Друк ризографія.
Ум. друк. арк. _____Наклад _____прим. Зам. №_____ Безкоштовно.

Редакційно-видавничий відділ
Кременчуцького національного університету
імені Михайла Остроградського
вул. Першотравнева, 20 м. Кременчук, 39600